

反思:语言代理 言语强化学习

诺亚·辛恩东北大学
noahshinn024@gmail.com

费德里科·卡萨诺东北大学
cassano.f@northeastern.edu

阿什温·戈皮纳特
麻省理工学院
agopi@mit.edu

卡尔蒂克·纳拉辛汉 普林斯顿大学
karthikn@princeton.edu

姚舜宇普林斯顿大学
shunyuy@princeton.edu

抽象的

大型语言模型 (LLM) 作为目标驱动型智能体,已被越来越多地用于与外部环境 (例如游戏、编译器、API) 进行交互。然而,由于传统的强化学习方法需要大量的训练样本和昂贵的模型微调,这些语言智能体如何快速高效地从试错中学习仍然是一个挑战。我们提出了一种名为 Reflexion 的新型框架,它并非通过更新权重,而是通过语言反馈来强化语言智能体。具体来说,Reflexion 智能体会对任务反馈信号进行口头反思,并将反思文本保存在情景记忆缓冲区中,从而在后续试验中做出更优的决策。Reflexion 具有足够的灵活性,可以整合各种类型 (标量值或自由语言) 和来源 (外部或内部模拟) 的反馈信号,并在各种任务 (序列决策、编码、语言推理) 中都取得了比基线智能体显著的改进。例如,Reflexion 在 HumanEval 编码基准测试中达到了 91% 的 pass@1 准确率,超越了之前最先进的 GPT-4 的 80%。我们还使用不同的反馈信号、反馈融合方法和智能体类型进行了消融和分析研究,并深入探讨了它们如何影响性能。所有代码、演示和数据集均发布在<https://github.com/noahshinn024/reflexion>。

1 引言

近期诸如 ReAct [30]、SayCan [1]、Toolformer [22]、HuggingGPT [23]、生成式智能体[19]和 WebGPT [17]等研究成果已证明,基于大型语言模型 (LLM) 核心构建的自主决策智能体是可行的。这些方法利用 LLM 生成文本和“动作”,这些文本和动作可用于 API 调用并在环境中执行。由于这些方法依赖于参数数量庞大的大型模型,因此目前只能使用上下文示例来训练智能体,因为像梯度下降强化学习这样的传统优化方案需要大量的计算资源和时间。

本文提出了一种名为“反思”(Reflexion)的替代方法,该方法利用语言强化来帮助智能体从先前的失败中学习。反思将来自环境的二进制或标量反馈转换为文本摘要形式的语言反馈,并将其作为附加上下文添加到下一回合中,供LLM智能体使用。这种自我反思的反馈充当“语义”梯度信号,为智能体提供具体的改进方向,帮助其从先前的错误中学习,从而更好地完成任务。这类类似于人类迭代学习的方式。



图 1:反思适用于决策 4.1、编程 4.3 和推理 4.2 任务。

通过反思之前的失败,为下一次尝试制定更完善的策略,从而以较少的尝试次数完成复杂任务。例如,如图 1 所示,Reflexion 智能体通过试错和自我反思,学习优化自身行为,以解决决策、编程和推理任务。

生成有用的反思性反馈极具挑战性,因为它需要深入了解模型出错之处(例如,信用分配问题[25]),以及生成包含可操作改进见解的摘要的能力。我们探索了三种方法:简单的二元环境反馈、针对常见故障情况的预定义启发式方法,以及自我评估,例如使用 LLM (决策)进行二元分类或使用自编写的单元测试(编程)。在所有实现中,评估信号都被放大为自然语言经验摘要,并可存储在长期记忆中。

与传统的强化学习方法(例如基于策略或基于价值的学习)相比,反思学习具有以下几个优势:1)它轻量级,无需对学习逻辑模型(LLM)进行微调;2)与难以准确分配积分的标量或向量奖励相比,它允许更细致的反馈形式(例如,针对性地改变行动);3)它允许使用比先前经验更明确、更易于解释的情景记忆;4)它为未来情景中的行动提供更明确的提示。与此同时,它也存在一些缺点,例如依赖于学习逻辑模型的自我评估能力(或启发式方法),并且没有正式的成功保证。然而,随着学习逻辑模型能力的提升,我们预期这种范式会随着时间的推移而不断改进。

我们开展了以下实验:(1) 决策任务,用于测试长轨迹上的序列动作选择;(2) 推理任务,用于测试知识密集型单步生成改进;(3) 编程任务,用于教会智能体有效使用编译器和解释器等外部工具。在所有三种类型的任务中,我们观察到 Reflexion 智能体在决策、推理和编程方面都表现更佳。更具体地说,Reflexion 智能体在 AlfWorld [24] 决策任务中,经过 12 个迭代学习步骤后,性能比强大的基线方法提高了 22%;在 HotPotQA [28] 推理问题中提高了 20%;在 HumanEval [6] Python 编程任务中提高了高达 11%。

总而言之,我们的贡献如下:

我们提出了 Reflexion,一种新的“语言”强化范式,它将策略参数化为智能体的记忆编码与 LLM 参数的选择相结合。

我们探索了 LLM 中自我反思的这种涌现特性,并通过经验表明,自我反思对于在少量试验中学习复杂任务非常有用。

我们推出了 LeetCodeHardGym,这是一个包含 40 个代码生成 RL 训练环境的训练营。19 种编程语言的 LeetCode 难题(“困难级别”)。

我们证明,Reflexion 在多个任务中都比强大的基线方法取得了改进,在各种代码生成基准测试中取得了最先进的结果。

相关推理和决策工作					
方法	自己	隐	决策二元记忆		
	完善约束条件,从而获得奖励				
自我精炼[15]	✓				
波束搜索[27]	✓				
反思（我们的）	✓		✓✓	✓✓	✓✓

相关编程工作					
方法	测试	调试自生成的多重自反射		语言	
测试执行	执行	测试			
字母代码[14]	✓			✓	
CodeT [5]	✓		✓		
自调试[7]	✓				
CodeRL [12]	✓				
反思（我们的）	✓		✓✓✓	✓	✓

2 相关工作

推理和决策自我改进[15]采用迭代框架进行自我改进,通过自我评估自主改进生成过程。这些自我评估

自我提升的步骤取决于既定的任务约束,例如“如何以更积极的方式书写这一代人的故事”。自我完善虽然有效,但仅限于单人。

推理任务。Pryzant 等人[21]执行类似的语义提示编写优化,但此外,还仅限于单代任务。Paul 等人[20]对评论家模型进行微调,以提供中间结果。

在轨迹中引入反馈以改进推理响应。Xie 等人[27]使用随机束通过搜索各种操作,执行更高效的决策搜索策略,从而实现:代理利用其自我评估成分的预见优势。Yoran 等人[31]和 Nair 等人。[16]使用决策模型进行多代推理。Kim 等人[10]使用重试模式。

固定步骤数,不包含评估步骤。Goodman [9]进行定性评估该步骤旨在对上一代算法进行优化。在本文中,我们展示了几个……

这些概念可以通过自我反思来强化,从而建立起持久的自我反思记忆。经验使代理人能够识别自身的错误,并自我总结经验教训。

随着时间的推移,会犯一些错误。

编程方面,过去和最近的一些作品采用了测试驱动开发的变体或

代码调试实践。AlphaCode [14]对一组隐藏的测试用例进行评估。CodeT [5]使用自行生成的单元测试来对生成的函数实现进行评分。

自调试[7]采用调试组件来改进现有实现。

CodeRL [12]将问题置于强化学习框架中,使用 Actor-Critic 模型,根据代码执行环境的反馈来调试程序。

AlphaCode、自调试和CodeRL在修复不太复杂的程序错误方面很有效,但是它们

依赖于那些会使 pass@1 资格失效的真实测试用例,并且不使用自我反思。

弥合错误识别和实施改进之间的差距。CodeT 不访问

虽然有隐藏的测试用例,但没有实现自学习步骤来改进代码编写。

3. 反思:通过言语反思进行强化

我们为 Reflexion 开发了一种模块化公式,利用了三个不同的模型:一个 Actor,表示为

Ma负责生成文本和动作; Me代表评估模型,负责对输出结果进行评分。

由马制作;以及一种自我反思模型(记为Msr),该模型可产生言语强化。

帮助演员自我提升的提示。我们对每一条提示都进行了详细描述。

模型,并随后阐明它们在反思框架内的协作功能。

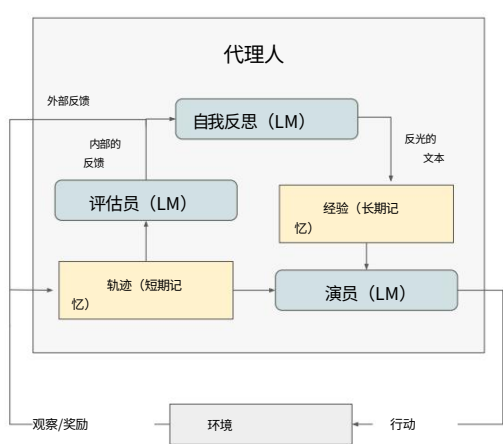
Actor角色构建于大型语言模型(LLM)之上,该模型专门用于提示

根据状态观测结果生成必要的文本和操作。类似于传统方法。

在基于策略的强化学习设置中,我们在时间t从当前策略 π_θ 中采样一个动作或生成,

接收来自环境的观察结果。我们探索各种 Actor 模型,包括链式模型。

思考[26]和反应[30]。这些不同的生成模型使我们能够探索不同的方面。



算法 1通过自我反思进行强化

初始化行动者、评估者和自我反思者：
马,我,先生
初始化策略 $\pi_\theta(a_i | s_i)$, $\theta = \{Ma, mem\}$
使用 π_θ 生成初始轨迹
使用 Me_i 评估 τ_0
使用 Ms_r 生成初始自反射 sr_0
设置内存 $\leftarrow [sr_0]$
设置 $t = 0$

while Me not pass or $t < \max trials$ do oi using π_θ
 生成 $\tau_t = [a_0, o_0, \dots a_i]$ 使用 Me_i 计算 τ_t 使 ,
 用 Ms_r 生成自反射 srt 将 srt 添加到 mem 递增 t 结束 while

返回

图 2: (a)反射示意图。(b)反射强化算法

本文研究了 Reflexion 框架内的文本和动作生成,从而深入了解其性能和有效性。此外,我们还添加了一个记忆组件 `mem`,为该智能体提供额外的上下文信息。这一改进借鉴了 Brooks 等人[3] 的研究,他们提出了一种基于上下文学习的策略迭代方法。下文将详细介绍其具体实现方式。

评估器 (Evaluator)是反思框架 (Reflexion framework)中的组件,它在评估行动者 (Actor)生成的输出质量方面发挥着至关重要的作用。它以生成的轨迹作为输入,并计算一个奖励分数,该分数反映了其在给定任务环境中的表现。定义适用于语义空间的有效价值和奖励函数十分困难,因此我们研究了评估器模型的几种变体。对于推理任务,我们探索了基于精确匹配 (EM)评分的奖励函数,以确保生成的输出与预期解决方案高度一致。在决策任务中,我们采用了预定义的启发式函数,这些函数针对特定的评估标准进行了定制。此外,我们还尝试使用 LLM 本身的不同实例作为评估器,为决策和编程任务生成奖励。这种多方面的评估器设计方法使我们能够检验不同的输出评分策略,从而深入了解它们在各种任务中的有效性和适用性。

自我反思模型以逻辑学习模型 (LLM)的形式实现,在反思框架中扮演着至关重要的角色,它通过生成语言形式的自我反思,为未来的试验提供有价值的反馈。给定稀疏的奖励信号 (例如二元成功状态 (成功/失败))、当前轨迹及其持久记忆,自我反思模型能够生成细致入微且具体的反馈。

这种反馈比标量奖励更具信息量,随后会被存储在智能体的记忆 (mem)中。例如,在多步骤决策任务中,当智能体收到失败信号时,它可以推断出某个特定动作 a_i 导致了后续的错误动作 a_{i+1} 和 a_{i+2} 。然后,智能体可以口头表达它本应采取不同的动作 a 和 a_{i+2} 。

我, 这将导致 $i+1$
并将此经验存储在记忆中。在随后的试验中,智能体可以利用过去的经验,在时间 t 通过选择行动 a 来调整其决策方法。这种反复试验、纠错、自我反思和持续记忆的过程,使智能体能够利用信息丰富的反馈信号,在各种环境中快速提升其决策能力。

反思过程的核心组成部分是短期记忆和长期记忆的概念。在推理阶段,行动者会根据短期记忆和长期记忆来做出决策,这类似于人类既能记住近期细节,又能从长期记忆中回忆起重要的经验教训。在强化学习 (RL)模型中,轨迹历史充当短期记忆,而自我反思模型的输出则存储在长期记忆中。这两个记忆组成部分协同工作,提供既具体又受经验教训影响的背景信息。

通过多次试验进行学习,这是 Reflexion 智能体相对于其他 LLM 动作选择工作的关键优势。

反思过程在式1中被形式化为一个迭代优化过程。在第一次试验中,行动者通过与环境交互生成轨迹 τ_0 。评估者随后生成一个分数 r_0 ,其计算公式为 $r_t = \text{Me}(\tau_0)$ 。 r_t 仅是第 t 次试验的标量奖励,随着特定任务表现的提高而增加。第一次试验后,为了将 r_0 放大为可供学习学习模型 (LLM)改进的反馈形式,自我反思模型分析集合 $\{\tau_0, r_0\}$,生成一个汇总信息 s_r ,并将其存储在内存 mem 中。 s_r 是第 t 次试验的口头经验反馈。行动者、评估者和自我反思模型在一个循环中协同工作,直到评估者认为 τ_t 正确为止。如式3所述,反思过程的内存组件对其有效性至关重要。每次试验 t 后, s_r 都会被添加到 mem 中。在实践中,我们通过存储经验的最大数量 Ω (通常设置为 1-3)来限制 mem ,以遵守最大上下文 LLM限制。

4个实验

我们评估了多种自然语言强化学习 (RL)设置,并考察了它们在决策、推理和代码生成任务上的性能。具体来说,我们让智能体在 HotPotQA [28] 上执行基于搜索的问答任务,在 AlfWorld [24] 上执行常见家庭环境中的多步骤任务,并在 HumanEval [6]、MBPP [2]和 LeetcodeHard (一个新的基准测试)等类似竞赛的环境中执行代码编写任务 (这些环境需要解释器和编译器)。值得注意的是,Reflexion在 AlfWorld、HotPotQA 和 HumanEval 上分别比强大的基线模型提升了 22%、20% 和 11% 的性能。

4.1 顺序决策:ALFWorld

AlfWorld 是一套基于文本的交互式环境,它要求智能体在基于 TextWorld [8]的各种交互式环境中完成多步骤任务。我们参考 Yao 等人[30]的方法,在 134 个 AlfWorld 环境中运行智能体,完成六项不同的任务,包括寻找隐藏物体 (例如,在抽屉里找到锅铲)、移动物体 (例如,将刀移动到砧板上)以及用其他物体操作物体 (例如,将西红柿放入冰箱冷藏)。我们使用ReAct [30]作为动作生成器,因为 Yao 等人[30]已证明 ReAct 能够利用显式中间思维进行长轨迹决策。AlfWorld 任务自然需要一个自我评估步骤,因为环境只能发出任务完成的信号。为了实现完全自主的行为,我们实现了两种自我评估技术:使用 LLM 的自然语言分类和手写启发式方法。启发式方法很简单:如果智能体连续 3 个周期执行相同的动作并收到相同的响应,或者当前环境中执行的动作数量超过 30 个 (规划效率低下),则进行自我反思。在基准运行中,如果系统建议进行自我反思,则跳过自我反思过程,重置环境并开始新的试验。在反思运行中,智能体使用自我反思来发现错误,更新记忆,重置环境并开始新的试验。为了避免提示窗口过长而超出最大限制,我们将智能体的记忆截断为最近 3 次自我反思 (经验)。

为了避免句法错误,我们为智能体提供了两条特定领域的少样本轨迹。我们使用了与 Yao 等人[30]针对 GPT-3 的 LLM相同的少样本轨迹示例。AlfWorld 任务、 ReAct 少样本提示和 Reflexion 示例包含在附录中。

结果显示,ReAct + Reflexion 的性能显著优于 ReAct,它利用简单的启发式方法来检测幻觉和低效的计划,成功完成了 134 项任务中的 130 项。此外,ReAct + Reflexion还通过连续 12 次试验学习解决更多任务。而在仅使用 ReAct 的方法中,我们发现性能提升在第 6 次和第 7 次试验之间停止。

分析:在基线失败的AlfWorld轨迹中,一个常见的错误是智能体误以为自己拥有某个物品,但实际上并没有。智能体会在较长的轨迹中执行多个动作,并且无法回溯这些动作来发现错误。反思机制通过自我反思,将冗长的失败轨迹提炼成相关的经验,并将其作为未来的“自我提示”,从而消除了几乎所有此类情况。在 AlfWorld中,长期记忆主要在以下两种情况下帮助智能体:1)可以轻松识别长轨迹中的早期错误。智能体可以建议新的动作选择,甚至新的长期计划。2)存在太多……

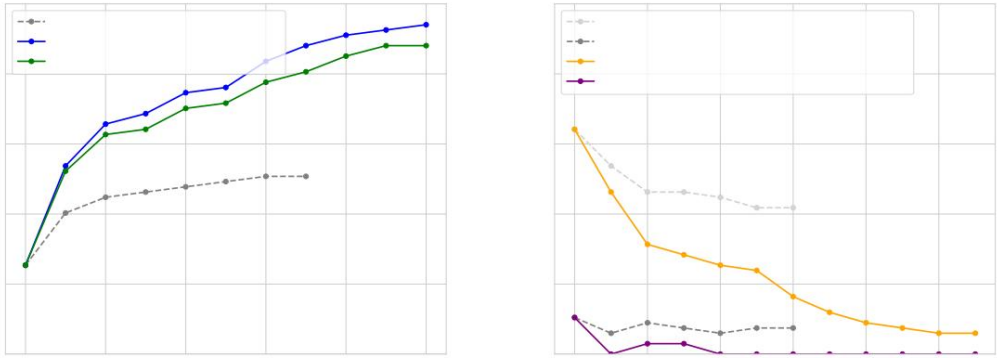


图 3: (a)AlfWorld 在 134 个任务中的表现,显示了使用 (启发式)和 (GPT)自评估技术进行二元分类时已解决任务的累积比例。(b)根据失败原因对 AlfWorld 轨迹进行分类。

需要检查许多表面/容器来寻找物品。智能体可以利用多次试验的经验记忆来彻底搜索房间。在图3中,学习曲线表明学习过程发生在多次经验中,这意味着智能体成功地平衡了案例1和案例2,这体现在前两次试验之间改进的立即飙升,然后在接下来的11次试验中稳步提升,最终达到近乎完美的表现。另一方面,图3显示,仅使用ReAct的智能体最终的幻觉率高达22%,且没有长期恢复的迹象。

4.2 推理:HotpotQA

HotPotQA [28]是一个基于维基百科的数据集,包含 11.3 万个问答对,旨在挑战智能体解析内容并推理多个支持文档。为了测试仅推理能力的提升,我们实现了 Reflexion + Chain-of-Thought (CoT) [26],分别用于逐步实现 $Q \rightarrow A$ 和 $Q, Cgt \rightarrow A$ 两种情况,其中 Q 是问题, Cgt 是来自数据集的真实上下文, A 是最终答案。由于 CoT 并非多步骤决策技术,我们将 Cgt 提供给智能体,以便能够隔离智能体在所提供文本的大部分区域上的推理行为。为了测试需要推理和行动选择的整体问答能力,我们实现了一个 Reflexion + ReAct [30]智能体,该智能体可以使用维基百科 API 获取相关上下文,并通过逐步显式思考推断答案。对于 CoT 实现,我们使用 6 次提示;对于 ReAct,我们使用两步提示法;对于自我反思,我们也使用两步提示法。所有示例均可在附录中找到。

稳健地评估自然语言答案一直是自然语言处理领域的一个长期难题。因此,在试验间隙,我们使用基于环境的精确匹配答案评分,为智能体提供一个二元成功信号。每次试验后,我们采用自我反思循环来放大该二元信号,类似于AlfWorld中决策设置4.1,其记忆容量为3次经验。

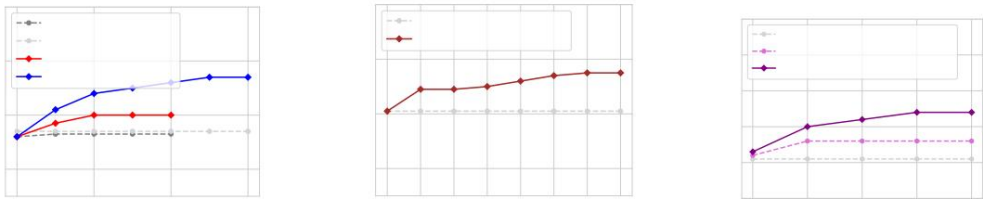


图 4:思维链 (CoT) 和 ReAct,Reflexion 提升了在 100 道 HotPotQA 问题上的搜索、信息检索和推理能力。(a) Reflexion ReAct 与 Reflexion CoT 的对比 (b) 仅用于推理的反射 CoT (GT) (c) 反射与情景记忆消融。

结果显示,在多个学习阶段,Reflexion 的性能显著优于所有基线方法。此外,仅使用 ReAct、仅使用 CoT 和仅使用 CoT (GT) 的实现方式均未能从概率上改进任何任务,这意味着在任何基线模型中,首次试验中没有失败的任务。

在后续试验中,使用 0.7 的温度,在反射运行中,这些问题得以解决。我们允许代理积累经验,并在失败的任务上反复尝试,直到它连续完成 3 次任务。在特定任务上尝试失败。自然而然地,CoT (GT)获得了更高的准确率,因为它已经获得了问题的真实上下文信息。然而,CoT (GT)智能体仍然无法……它能正确推断出39%问题的正确答案,但反思功能可以帮助智能体进行更正。

在没有获得真实答案的情况下,它犯了错误,导致准确率提高了 14%。

分析我们进行了一项消融实验,以找出自反射步骤的优势

以CoT (GT)作为基准方法进行推理 4. 回顾一下,CoT (GT)使用链式思维。利用提供的真实上下文进行推理,测试在较长上下文中的推理能力。接下来,我们通过纳入最近的轨迹,添加了情景记忆 (EPM)元素。对于对于反思代理,我们将标准的自我反思步骤作为最后一步实现。直观地说,我们测试……通过使用书面语言进行口头解释,智能体正在迭代地更有效地学习。

以第一人称视角。结果4表明,自我反思能使学习效果绝对提升8%。情景记忆学习优势。这一结果支持了仅进行精细化学习的论点。

这种方法不如自我反思引导的改进方法有效。

4.3 编程

我们评估了基准方法和 Reflexion 方法在 MBPP 平台上编写 Python 和 Rust 代码的性能。

[2].HumanEval [6] 和 LeetcodeHardGym (我们新的数据集) 。MBPP 和 HumanEval 指标基于自然语言描述的函数体生成准确率。我们使用基准语言。

MultiPL-E [4] 编译器用于将 HumanEval 和 MBPP 的子集翻译成 Rust 语言。MultiPL- E 是一组小型编译器,可用于将 Python 基准测试问题翻译成 18 个Rust 语言。

其他语言。我们加入了 Rust 代码生成的实验,以证明 Reflexion 的有效性。

代码生成的实现与语言无关,可用于解释型和交互式代码。

编译型语言。最后,我们引入了一个新的基准测试,LeetcodeHardGym,它是一个包含 40 道已发布的 LeetCode 难度较高的交互式编程训练题的交互式编程训练营

2022 年 10 月 8 日之后,这是 GPT-4 的预训练截止日期 [18]。

编程任务提供了一个独特的机会,让我们能够运用更务实的自我评估方法。

例如,自行生成的单元测试套件。因此,我们基于反射的编程任务实现是符合 pass@1 准确率报告要求。为了生成测试套件,我们使用链式思维提示法。

[26] 生成多样化、广泛的测试,并附上相应的自然语言描述。然后,我们通过尝试构建有效的抽象语法树来筛选语法上有效的测试语句。

(AST)针对每个提出的测试。最后,我们从生成的单元测试集合中抽取n 个测试样本。

生成一个测试套件T,记为{t0, t1, ..., tn}。我们将n设置为最多 6 个单元测试。此外从单元测试套件组件中,可以设置 Reflexion 编程代理的学习循环。

与最大记忆限制为 1 次经验的推理和决策代理相同。

基准 + 语言 先前 SOTA 通过@1人工评估 (PY)		SOTA Pass@1 Reflexion Pass@1	
HumanEval (RS)	65.8 (CodeT[5] + GPT-3.5)	80.1 (GPT-4)	91.0
	-	60.0 (GPT-4)	68.0
MBPP (PY)	67.7 (代码T [5] + 代码[6])	80.1 (GPT-4)	77.1
MBPP (RS)	-	70.9 (GPT-4)	75.4
Leetcode Hard (PY)	-	7.5 (GPT-4)	15.0

表 1:不同模型-策略-语言组合的 Pass@1 准确率。基础策略是
单次代码生成示例。所有基于指令的模型都遵循零样本代码生成方法。

结果显示,Reflexion 的性能优于所有基准准确率,并树立了新的尖端标准。

除 MBPP Python 1 外,所有 Python 和 Rust 基准测试均已完成。我们进一步调查了其不足之处。

Reflexion 在 MBPP Python 上的性能。

分析我们承认,具有自我反思能力的代码生成代理受限于其自身的能力
编写多样化、全面的测试。因此,如果模型生成不稳定的测试,则需要进行相应的调整。

基准 + 语言基础反射	TP	FN	FP	TN
HumanEval (PY)	0.80	0.91	0.80	0.99 0.40 0.01 0.60
MBPP (PY)	0.77	0.60	0.68	0.84 0.59 0.16 0.41
HumanEval (RS)	0.71	0.75		0.87 0.37 0.13 0.63
MBPP (RS)				0.84 0.51 0.16 0.49

表 2:HumanEval 和 MBPP 的总体准确率和测试生成性能。对于 Rust, HumanEval 是 HumanEval Python 中最难的 50 道题,已用 Rust 语言和 MultiPL-E 实现。
[4]. TP:单元测试通过,解决方案通过;FN:单元测试失败,解决方案通过;FP:单元测试通过,解决方案失败;TN:单元测试失败,解决方案失败。

在测试套件中,所有测试都可能通过了错误的验证,从而导致出现假阳性标签。代码补全[11]。另一方面,如果模型生成了编写错误的测试套件,则有些测试即使采用正确的解决方案也可能失败,从而引发自我反思。这取决于代码补全是否出现错误(假阴性)。鉴于 Reflexion 的实现方式,假阴性比假阳性更可取,因为代理人可能能够通过自我反思来避免这种情况。识别错误的测试用例,并提示自身保持原始代码补全功能不变。另一方面,如果无效的测试套件返回错误完成结果(所有内部测试用例均通过),则会导致测试套件完成失败。但如果实现不正确,代理会过早地报告无效提交。在第 2 节中,我们测量了各种条件,以分析除一次通过率之外的性能。此前,我们在 MBPP Python 测试中,Reflexion 的性能不如基线 GPT-4。在第 2 部分中,我们观察到内部测试执行产生的误报标签之间存在显著差异, P(非通过@1代正确 | 测试通过)。也就是说,给定给定值,提交失败的概率。它通过了所有单元测试。对于 HumanEval 和 MBPP Python,基准 pass@1 准确率如下: 两者相对接近,分别为 82% 和 80%。然而,假阳性测试执行率…… MBPP Python 的成功率为 16.3%,而 HumanEval Python 的成功率仅为 1.4%,两者总体成功率合计为 91%。 准确度 1.

方法	测试生成自我反思通过@1 (准确)		
基础型号	错误的	错误的	0.60
测试生成遗漏 错误		真的	0.52
自我反思缺失 真		错误的	0.60
反射	真的	真的	0.68

表 3:使用 Reflexion 方法的各种妥协方案的 Pass@1 准确率
HumanEval Rust 以 GPT-4 为基础模型 - 50 个最难问题

消融研究我们测试了 Reflexion 用于测试生成和自我反思的复合方法在 HumanEval Rust 问题中最难的 50 个问题子集中开展合作。我们的 Rust 编译器环境提供详细的错误日志和有用的调试提示,因此是一个很好的试验平台。对于妥协方案,首先,我们省略内部测试生成和执行步骤,这些步骤测试智能体在没有当前实现指导的情况下进行自我反思的能力。图3显示了较差的表现。准确率分别为 52% 和 60% (基线),这表明代理无法确定当前情况。没有单元测试,实现也是正确的。因此,代理必须参与所有迭代。运行过程中没有提前返回的选项,会对实现进行有害的修改。接下来,我们通过省略自然语言解释步骤来测试自我反思的贡献。单元测试套件评估失败。直观地说,这要求智能体将以下任务结合起来:对所有失败的单元测试进行错误识别和实现改进。有趣的是,受损代理的性能并未优于基准运行。我们观察到测试结果。代码生成和编译步骤能够捕获语法和逻辑错误,但实现过程却无法完全捕获这些错误。修复方案并未反映这些迹象。这些实证结果表明,最近的一些研究……提出不加反思的盲目试错调试技术,对更复杂的问题来说是无效的。例如用 Rust 编写复杂的程序。

5. 局限性

Reflexion 的核心是一种利用自然语言进行策略优化的优化技术。

策略优化是一种通过经验改进行动选择的有效方法,但它仍然可能陷入非最优的局部最小值。在本研究中,我们将长期记忆限制在一个具有最大容量的滑动窗口内,但我们鼓励未来的研究使用更高级的结构(例如向量嵌入数据库或传统的 SQL 数据库)来扩展 Reflexion 的记忆组件。具体到代码生成方面,测试驱动开发在指定精确的输入输出映射方面存在许多实际限制,例如非确定性生成器函数、与 API 交互的非纯函数、根据硬件规格改变输出的函数,或者调用难以预测的并行或并发行为的函数。

6. 更广泛的影响

大型语言模型越来越多地被用于与外部环境(例如互联网、软件、机器人等)和人类进行交互。我们的工作有望增强这些智能体的能力,使其实现更高的自动化水平和工作效率,但同时也加剧了这些智能体被滥用时的风险。我们认为,这一研究方向需要在安全性和伦理方面投入更多精力。

另一方面,强化学习由于其黑箱策略和优化设置,在可解释性和一致性方面存在诸多挑战。我们提出的“语言”强化学习或许能够解决部分问题,并使自主智能体更易于解释和诊断。例如,对于人类难以理解的工具使用情况,可以监控自主智能体在使用工具前的自我反思,以确保其意图正确。

7 结论

本文提出了一种名为“反思”(Reflexion)的方法,该方法利用语言强化来训练智能体从过去的错误中学习。我们通过实证研究表明,反思智能体通过自我反思显著优于目前广泛使用的决策方法。在未来的工作中,反思智能体可以应用于在传统强化学习环境中已得到充分研究的更高级技术,例如自然语言价值学习或离策略探索技术。

8. 可重复性

我们强烈建议其他人在运行自主代码编写实验时使用隔离的执行环境,因为生成的代码在执行前未经验证。

参考

- [1] Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., et al. (2022). Do as i can, not as i say: Grounding language in robotic affordables. arXiv preprint arXiv:2204.01691.
- [2] Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q. 等人。(2021)。使用大型语言模型进行程序综合。arXiv 预印本 arXiv:2108.07732。
- [3] Brooks, E., Walls, L., Lewis, RL 和 Singh, S. (2022)。上下文策略迭代。arXiv 预印本 arXiv:2210.03821。
- [4] Cassano, F., Gouwar, J., Nguyen, D., Nguyen, S., Phipps-Costin, L., Pinckney, D., Yee, M.-H., Zi, Y., Anderson, C.J., Feldman, M.Q., Guha, A., Greenberg, M., 和 Jangda, A. (2022). Multipl-e: 一种可扩展的神经代码生成基准测试方法。
- [5] Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G., and Chen, W. (2022). Codet: Code 使用生成的测试进行生成。arXiv 预印本 arXiv:2207.10397。

- [6] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, HP d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021). Evaluating large language models training on code. arXiv preprint arXiv:2107.03374.
- [7] Chen, X., Lin, M., Schärli, N., and Zhou, D. (2023). Teaching large language models to self-debug. arXiv preprint arXiv:2304.05128.
- [8] Côté, M.-A., Kádár, A., Yuan, X., Kybartas, B., Barnes, T., Fine, E., Moore, J., Hausknecht, M., El Asri, L., Adada, M., et al. (2019). Textworld: 一个基于文本的游戏学习环境。载于《计算机游戏: 第七届研讨会, CGW 2018, 与第27届国际人工智能大会, IJCAI 2018 同期举办, 瑞典斯德哥尔摩, 2018年7月13日, 修订版精选论文集7》, 第41-75页。Springer出版社。
- [9] Goodman, N. (2023). Meta-prompt: 一个简单的自改进语言代理。noahgoodman.substack.com。
- [10] Kim, G., Baldi, P., and McAleer, S. (2023). 语言模型可以解决计算机任务。arXiv预印本 arXiv:2303.17491。
- [11] Lam, W., Winter, S., Wei, A., Xie, T., Marinov, D., 和 Bell, J. (2020). 片状测试的大规模纵向研究。Proc. ACM Program. Lang., 4(OOPSLA)。
- [12] Le, H., Wang, Y., Gotmare, AD, Savarese, S., 和 Hoi, SCH (2022). Coderl: 通过预训练模型和深度强化学习掌握代码生成。神经信息处理系统进展, 35:21314–21328。
- [13] Li, R., Allal, LB, Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., 等人。(2023)。StarCoder: 愿源头与你同在! arXiv 预印本 arXiv:2305.06161。
- [14] Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. (2022). 使用 alphacode 进行竞赛级代码生成。科学, 378(6624):1092–1097。
- [15] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhume, S., Yang, Y. 等人。(2023)。自我细化: 通过自我反馈进行迭代细化。arXiv预印本 arXiv:2303.17651。
- [16] Nair, V., Schumacher, E., Tso, G., 和 Kannan, A. (2023). Dera: 使用支持对话的解析代理增强大型语言模型补全。arXiv 预印本 arXiv:2303.17071。
- [17] Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al. (2021). Webgpt: 浏览器辅助的带人工反馈的问答系统。arXiv预印本 arXiv:2112.09332。
- [18] OpenAI (2023)。Gpt-4 技术报告。ArXiv。
- [19] Park, JS, O'Brien, JC, Cai, CJ, Morris, MR, Liang, P., 和 Bernstein, MS (2023)。生成式智能体: 人类行为的交互式模拟。arXiv 预印本 arXiv:2304.03442。
- [20] Paul, D., Ismayilzada, M., Peyrard, M., Borges, B., Bosselut, A., West, R., 和 Faltings, B. (2023). Refiner: 中间表示的推理反馈。arXiv 预印本 arXiv:2304.01904。
- [21] Pryzant, R., Iter, D., Li, J., Lee, YT, Zhu, C., 和 Zeng, M. (2023)。使用梯度下降和束搜索的自动即时优化。arXiv 预印本 arXiv:2305.03495。
- [22] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., 和 Scialom, T. (2023)。Toolformer: 语言模型可以自学使用工具。arXiv预印本 arXiv:2302.04761。
- [23] Shen, Y., Song, K., Tan, X., Li, D., Lu, W., and Zhuang, Y. (2023). Hugginggpt: Solving ai tasks with chatgpt and its friends in huggingface. arXiv preprint arXiv:2303.17580.

- [24] Shridhar, M., Yuan, X., Côté, M.-A., Bisk, Y., Trischler, A. 和 Hausknecht, M. (2021)。ALFWorld:将文本和具身环境相结合以实现交互式学习。载于国际学习表征会议 (ICLR)论文集。
- [25] Sutton, RS 和 Barto, AG (2018)。强化学习:导论。麻省理工学院出版社,第二版。
- [26] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., and Zhou, D. (2022). Chain of thought prompting elicits reasoning in large language models. arXiv preprint arXiv:2201.11903.
- [27] Xie, Y., Kawaguchi, K., Zhao, Y., Zhao, X., Kan, M.-Y., He, J., and Xie, Q. (2023). 分解通过自评估引导解码增强推理。arXiv 预印本 arXiv:2305.00633。
- [28] Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, WW, Salakhutdinov, R. 和 Manning, CD (2018)。HotpotQA:一个用于多样化、可解释的多跳问答的数据集。在自然语言处理实证方法会议 (EMNLP)上。
- [29] Yao, S., Chen, H., Yang, J., 和 Narasimhan, K. (预印本)。网上商店:迈向可扩展利用基于语境的语言代理进行现实世界的网络交互。发表于 ArXiv。
- [30] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In International Conference on Learning Representations (ICLR).
- [31] Yoran, O., Wolfson, T., Bogin, B., Katz, U., Deutch, D., 和 Berant, J. (2023)。通过对多个思维链进行元推理来回答问题。arXiv 预印本 arXiv:2304.13007。

使用其他模型进行评估

我们进一步研究了试错法问题解决方法在各种模型中的适用性。优势。我们发现,能够明确自我纠正的能力是更强者的涌现特质,更大型号。

方法通过率 (8 次试验的平均值)	通过率 (标准差)
基线 0.26 反射 0.26	0.00481 0.00305

表 4:使用 starhat-beta [13] 在 HumanEval Python 上 Pass@1 准确率。

模型	基线准确率	反射准确率
CoT (GT) + text-davinci-003 0.60	CoT (GT) + gpt-3.5-turbo 0.57	0.77
CoT (GT) + gpt-4 0.68	ReAct + text-davinci-003 0.30	0.71
ReAct + gpt-3.5-turbo 0.26	ReAct + gpt-4 0.39	0.80
表 5:使用各种模型对 100 个 HotPotQA 进行 Pass@1 准确率测试。		0.55
		0.38
		0.51

B 决策

环境:你身处房间中央。快速环顾四周,你看到一张床 1、一张桌子2、一张桌子 1、一个抽屉 6、一个抽屉 5、一个抽屉 4、一个抽屉 3、一个抽屉 2、一个抽屉 1、一个垃圾桶 1、一个洗衣篮 1、一个保险箱 1、一个架子 6、一个架子 5、一个架子 4、一个架子 3、一个架子 2 和一个架子 1。

任务:用台灯检查杯子。
尝试 1> 思

考:为了完成任务,我需要找到并拿走一个杯子,然后再找到并使用一盏台灯。>思考:首先我需要找到一个杯子。杯子最有可能出现在抽屉 (1-6)、桌子 (1-2)、架子 (1-6)、垃圾桶 (1)或洗衣篮 (1)中。我可以逐一检查,从抽屉 1 开始。> 前往抽屉 1 抽屉 1 是关着的。

.....

> 打开抽屉 6 你打开了抽
屉 6。抽屉 6 是打开的。里面有一个钥匙扣 2。> 走到桌子 1 在桌子 1 上,你看到一张信用卡 3、一盏台灯 1、一
台笔记本电脑 2、一个
马克杯 1、一支钢笔 1 和一支铅笔 1。> 想:现在我找到了一个马克杯 (1)。接下来,我需要把它拿走。> 从桌子 1 上拿起马克杯 1 你从桌子 1 上
拿起马克杯 1。>想:现在我拿走了一个马克杯 (1)。接下来,我需要找到一盏台灯。
台灯更有可能出现在桌子 (1-2)、边桌
(1-2)、书架 (1-6)、床 (1) 和抽屉 (1-6) 中。我可以逐一检查,
从桌子 1 开始。> 走到桌子 1 什么也没发生。> 走到桌子 2 上。在桌子 2 上,你看到闹钟 1、碗 1、杯子 3、铅笔 3 和铅笔 2。> 想:现在我找到一盏台灯
(1)。接下来,我需要使用它。> 使用台灯 1。什么也没发生。> 使用台灯 1。什么也没发生。

状态:失败反思:
在这个环境中,我的计划是先找到一个杯子,然后再找到并使用台灯。然而,任务要求是用台灯检查杯子。我应该先找到台灯,然后再找杯子。我注意到台灯
在1号桌子上。在下次尝试中,我会先去1号桌子找到台灯,然后再找到杯子并用台灯检查它。

第二次尝试
> 走到桌子 1 上。在桌
子 1 上,你看到一张信用卡 3、一盏台灯 1、一台笔记本电脑 2、一个马克杯 1、一支钢笔 1 和一支铅笔 1。> 思考:为了完成任务,我需要找到并拿走
一个马克杯,然后再找到并使用一盏台灯。> 从桌子 1 上拿走马克杯 1。你从桌子 1 上拿起马克杯 1。> 思考:为了完成任务,我需
要找到并拿走一个马克杯,然后再找到
并使用一盏台灯。> 使用台灯 1。你打开台灯 1。

状态:成功

图 5:[上图] 智能体由于规划效率低下而失败的 AlfWorld 轨迹。在反思过程中,智能体意识到它应该先寻找台灯再寻找杯子,而不是先寻找杯子再寻找台灯。[下图] 智能体能够修正其推理轨迹,并以简洁的方式执行一系列动作。

B.1 网店限制

在第5点中,我们简要指出,Reflexion难以克服那些需要极高精度的局部最小值选择。为了逃避现实而表现出的创造性行为。我们在WebShop的一项实验中观察到了这一缺陷[29]。WebShop是一个基于Web的问题解决基准测试,用于测试代理在电子商务环境中导航的能力。该网站用于根据客户需求查找和购买产品。我们测试了两种 ReAct+ 组合。在 100 个环境中测试 Reflexion 智能体。然而,仅进行了四次试验后,我们就终止了运行。6. 代理未表现出任何改进迹象。此外,代理无法生成有用、直观的信息。失败尝试后的自我反思。我们得出结论:反思能力无法解决需要……的任务。大量的多样性和探索性。在AlfWorld中,智能体能够充分地探索由于可以从观察结果中看出允许的行为,因此可以适应新的环境。在 HotPotQA 中,该代理面临着类似的网上商店搜索查询任务,但由于其搜索空间更大,因此更为成功。维基百科的文章种类繁多,对搜索查询的精确度要求也较低。这是一个常见的问题。电子商务搜索引擎能够妥善处理自然语言搜索解释中的歧义。因此,WebShop 提出了一项需要 Reflexion 代理表现出非常多样化和独特行为的任务。

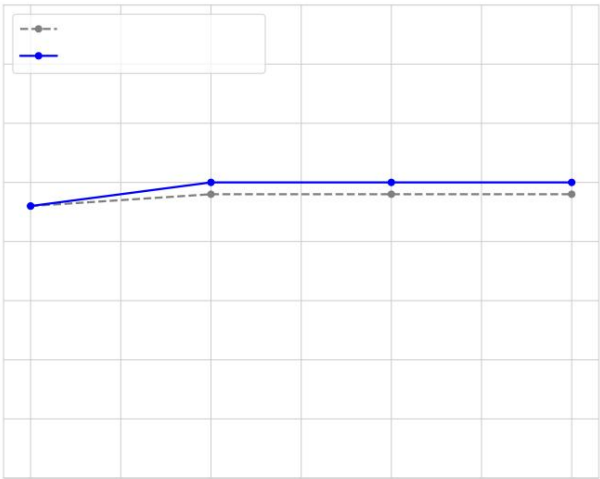


图 6:Reflexion 与 React 在 WebShop 上处理 100 个客户购物请求的性能对比。
ReAct + Reflexion 的性能并未显著优于 ReAct。

C语言编程

由于以下原因,LLM 调用编程需要严格的指令,仅生成函数体:
对LLM学生进行了大量的对话训练。以下列举几个编程示例。
说明以蓝色高亮显示,并提供模板。完整实现请参见<https://github.com/noahshinn024/reflexion>。

C.1 编程函数实现示例 (HumanEval Python)

函数签名示例:

```
1 def minSubArraySum ( nums ) :  
2  
3     给定一个整数数组 nums,求其中所有元素的最小和。  
4     任何  
5     非空数字子数组。  
6     例子  
7     minSubArraySum([2, 3, 4, 1, 2, 4]) == 1
```

```

7 minSubArraySum([-1, -2, -3]) == -6
8

```

函数体实现示例：

```

1 min_sum = float( inf )
2 for i in range( len (nums)):
3     当前总和 = 0
4     for j in range( i len (nums)): ,
5         current_sum += nums [j]
6         如果current_sum < min_sum:
7             最小总和 = 当前总和
8     返回最小和

```

C.2 反思演员指导

你是一名Python编程助手。你会得到你之前实现的一个函数，一系列单元测试结果，以及你对之前实现的反思。应用仅需对以下函数进行必要的修改，无需包含以下内容。请在回复中签名。回复的第一行应缩进 4 个空格。它与用户提供的签名在语法上相符。您将会收到一些示例。

用户。

反射演员的世代遵循以下形式：

- （操作说明）
- （功能实现）
- （单元测试反馈）
- （自我反思）
- （下一个函数实现的说明）

完整的示例请参见<https://github.com/noahshinn024/reflexion>

C.3 反思 自我反思指导和示例

你是一名Python编程助手。你会得到你之前实现的一个函数，一系列单元测试结果，以及你对之前实现的反思。应用仅需对以下函数进行必要的修改，无需包含以下内容。请在回复中签名。回复的第一行应缩进 4 个空格。它与用户提供的签名在语法上相符。您将会收到一些示例。

用户。反思自我反思的生成遵循以下形式：

- （操作说明）
- （功能实现）
- （单元测试反馈）

C.4 反思编程 无自我反思消融示例

反思 无自我反思 消融 行动者世代遵循以下形式：

- （操作说明）
- （功能实现）
- （单元测试反馈）
- （自我反思）
- （下一个函数实现的说明）

C.5 反射编程无测试生成消融示例

反思无测试生成消融 Actor 生成遵循以下形式：

- （操作说明）

(功能实现)
~~(单元测试反馈)~~
(自我反思)
(下一个函数实现的说明)

D 推理

D.1 完整示例

试题 1:问题:电影《长大成人》的主演是哪位演员?他最著名的角色是电视剧《Allo Allo!》中的哪个角色?

想法 1:我需要搜索《长大成人》和《Allo Allo!》,找到出演《长大成人》的演员,然后找到他在《Allo Allo!》中最著名的角色。

操作 1:搜索[成年人]

观察 1:《长大成人》(Grown-Ups)是1980年英国BBC出品的一部电视电影,由迈克·李构思并执导。主演包括莱斯利·曼维尔、菲利普·戴维斯、布伦达·布萊斯、珍妮·杜维茨基、琳赛·邓肯和萨姆·凯利。该片由罗宾·塞尔斯剪辑,路易斯·马克斯担任制片人,并于1980年11月28日在BBC 2频道首播。

想法二:《长大成人》的主演是莱斯利·曼维尔、菲利普·戴维斯、布伦达·布萊斯、珍妮·杜维茨基、琳赛·邓肯和萨姆·凯利。我需要搜索“Allo Allo!”,找出哪位演员最知名的角色。

操作 2:搜索[Allo Allo!]

观察 2:找不到[Allo Allo!]
类似:[“Allo Allo!”、“Allo Allo!角色列表”、Gorden Kaye、“Allo Allo!(系列2)”、Allô,allô、Francesca Gonshaw、“Allo Allo!(系列4)”、Kim Hartman、“Allo Allo!(系列6)”,杰里米·劳埃德]

想法 3:要找到因在《Allo Allo!》中扮演哪个角色而闻名的演员,我可以搜索Gorden Kaye。

行动 3:搜索[戈登·凯]

观察 3:戈登·欧文·凯(Gordon Irving Kaye,1941年4月7日-2017年1月23日),艺名戈登·凯(Gorden Kaye),是一位英国演员,因在电视喜剧系列剧《Allo Allo!》中扮演风流成性的咖啡馆老板雷内·阿尔图瓦(René Artois)而闻名。

想法四:戈登·凯最著名的角色是在电视喜剧系列剧《Allo Allo!》中扮演风流成性的咖啡馆老板雷内·阿尔图瓦。所以答案是雷内·阿尔图瓦。

动作 4:完成[勒内·阿尔图瓦]

观察 4:答案错误。反思:我搜索的剧名“Allo Allo!”有误,导致没有结果。我应该搜索剧中主角戈登·凯(Gorden Kaye)的名字,才能找到他最知名的角色。

试题二:问题:电影《长大成人》的主演是哪位演员?他最著名的角色是电视剧《Allo Allo!》中的哪个角色?

想法 1:我需要搜索一下在电视剧《Allo Allo!》中最著名的演员,并找出他最著名的角色是什么。

操作 1:搜索[成年人]

观察1:《长大成人》(Grown-Ups)是1980年英国BBC出品的一部电视电影,由迈克·李构思并执导。主演包括莱斯利·曼维尔、菲利普·戴维斯、布伦达·布萊斯、珍妮·杜维茨基、琳赛·邓肯和萨姆·凯利。该片由罗宾·塞尔斯剪辑,路易斯·马克斯担任制片人,并于1980年11月28日在BBC 2频道首播。

想法二:这段文字没有提到那位因出演《Allo》而闻名的演员。

“喂!”我需要搜索的是演员的名字。

行动 2:搜索[Sam Kelly]

观察2:罗杰·迈克尔·凯利(Roger Michael Kelly, 1943年12月19日—2014年6月14日),艺名萨姆·凯利(Sam Kelly),是一位英国演员,活跃于电影、电视、广播和戏剧界。他最著名的角色包括《Allo Allo!》中的汉斯·吉尔上尉(Captain Hans Geering)、《Porridge》中的沃伦(Warren)、《On the Up》中的萨姆(Sam)以及《Barbara》中的泰德·利弗里奇(Ted Liversidge)。

想法 3:萨姆·凯利最出名的角色是《Allo Allo!》中的汉斯·吉林上尉,所以答案是汉斯·吉林上尉。

行动 3:完成[汉斯·吉尔队长]

观察结果 3:答案正确

图 7:在相同环境和任务下进行的两次 HotPotQA 试验。Reflexion + ReAct 智能体利用自我反思来确定下一次试验中更优的搜索方法。

D.2 思维链 + 反思

试题一:问题:约翰·兰彻斯特和艾伦·迪恩·福斯特的共同职业是什么?

思路一:让我们一步一步来。约翰·兰彻斯特是一位小说家、记者和评论家。艾伦·迪恩·福斯特是一位小说家和编剧。因此,约翰·兰彻斯特和艾伦·迪恩·福斯特的共同职业是小说家和编剧。

试题二:问题:约翰·兰彻斯特和艾伦·迪恩·福斯特的共同职业是什么?

思路一:让我们一步一步来。约翰·兰彻斯特是一位小说家、记者和财经记者。艾伦·迪恩·福斯特是一位小说家和编剧。所以,约翰·兰彻斯特和艾伦·迪恩·福斯特的共同职业是小说家。

行动 1:完成[小说家和编剧]

观察1:答案错误。反思:我对约翰·兰彻斯特和艾伦·迪恩·福斯特共同职业的推理失败了,因为我错误地假设他们从事相同的职业。以后在解答这类问题时,我应该更深入地研究两位作者的个人背景,以确保准确识别他们的职业。此外,我还应该考虑到他们可能从事多种共同职业。

行动 1:完成[小说家]

观察结果 1:答案正确

D.3 HotPotQA 思维链 (GT) + 反思

背景:白原战役是美国独立战争期间纽约和新泽西战役中的一场战役,于1776年10月28日在纽约州白原附近爆发。撤退之后……

...

试题 1:问题: 1776 年 10 月 28 日,在纽约州白原市附近爆发了一系列争夺纽约市和新泽西州控制权的战役,这些战役被称为什么?

思路一:题目问的是美国独立战争期间,为争夺纽约市和新泽西州的控制权而爆发的一系列战役,其中一场于1776年10月28日在纽约州白原市附近爆发。根据上下文,我知道这就是白原战役。

试题二:问题: 1776 年 10 月 28 日,在纽约州白原市附近爆发了一系列争夺纽约市和新泽西州控制权的战役,这些战役被称为什么?

思路一:题目要求的是1776年10月28日在纽约州白原市附近爆发的一系列战役,争夺纽约市和新泽西州的控制权。根据上下文,我知道这一系列战役就是纽约和新泽西战役。

行动 1:结束[白原之战]

观察结果1:答案错误

行动 1:完成[纽约和新泽西运动]

观察结果 1:答案正确

反思:反思我之前给出的错误答案后,我意识到我可能没有提供足够的背景信息来准确回答问题。题目要求列举一系列战役,但我只提到了其中一场战役的名称。为了以后更好地回答这类问题,我会确保提供更多背景信息,例如战役名称 纽约和新泽西战役,以及它是一系列战役的事实。此外,我还会确保包含战役的日期和地点,以确保我的答案全面准确。

D.4 HotPotQA情景记忆 (EPM)消融提示

D.4.1 (EPM) 思维链 + 反思

试题一:问题:Jonny Craig 和 Pete Doherty 谁曾是更多乐队的成员?

思路一:我们一步一步来。Jonny Craig 曾是六支乐队的成员,包括 Emarosa、Dance Gavin Dance 和 Slaves。Pete Doherty 曾是七支乐队的成员,包括The Libertines、Babyshambles 和 The Puta Madres。所以 Pete Doherty 曾是更多乐队的成员。

动作 1:结束[皮特·多赫蒂]
观察结果1:答案错误

试题二:问题:Jonny Craig 和 Pete Doherty 谁曾是更多乐队的成员?

思路一:我们一步一步来。我会重新查阅这两位音乐人过去和现在的乐队经历,以确保我能准确比较他们的乐队成员情况。经过研究,我发现 Jonny Craig 曾是七个乐队的成员,包括 Emarosa、Dance Gavin Dance 和 Slaves。Pete Doherty 也曾是七个乐队的成员,包括 The Libertines、Babyshambles 和 The Puta Madres。所以 Jonny Craig 曾是更多乐队的成员。

动作 2:终结[强尼·克雷格]
观察结果 2:答案正确

反思:我之前关于哪位音乐人曾加入过更多乐队的推理失败了,因为我没有考虑到Jonny Craig过去曾加入过更多乐队,尽管Pete Doherty目前加入的乐队更多。以后在回答这个问题时,我应该重点研究这两位音乐人过去和现在的乐队,以确保我能准确地比较他们的乐队成员情况。

D.4.2 (EPM) 思维链 (GT) + 反思

背景:哈里·巴哈杜尔·巴斯内特是尼泊尔政治家,现任尼泊尔民族力量党外交关系部部长。巴斯内特拥有工程学硕士学位。
理学硕士 (拉丁语:Magister Scientiae;缩写为MS、MSc、M.Sc.、MSci、M.Sci.、 ScM、Sc.M.、SciM或Sci.M.)是科学领域的硕士学位……

试题 1:问题:国家人民力量党外交关系部负责人拥有的学位可以缩写为 MS、MS 或 ScM,请问该学位属于哪个领域?

(先前轨迹)
想法一:问题是询问国家人民力量党外交关系部部长拥有哪个领域的学位。根据上下文,我知道他拥有的是理学硕士学位,该学位通常授予科学、工程和医学领域的学生。

行动 1:完成[科学、工程和医学]
观察结果1:答案错误

试题二:问题:国家人民力量党外交关系部负责人拥有的学位可以缩写为 MS、MS 或 ScM,请问该学位属于哪个领域?

(先前轨迹)
想法一:题目问的是国家人民力量党外交关系部部长拥有哪个专业的学位。根据上下文,我知道他拥有的是理学硕士学位,该学位通常授予科学、工程和医学领域的学生。然而,题目问的是专业领域,而不是学位。

根据上下文,我知道 Hari Bahadur Basnet 拥有工程学硕士学位。
行动 1:完成[工程]
观察结果 1:答案正确

反思:第一次尝试时,我误解了题目。我以为题目问的是学位类别,而不是具体的学位。我吸取了教训,在第二次尝试中,我更加仔细地阅读题目,理解了它问的是一个具体的学科领域。